

Kotlin + Spring Boot

基礎からWebアプリケーションの処理の流れまで

Kotlin / Spring Boot / Spring MVC / Thymeleaf / Spring Data JPA / PostgreSQL / REST API

この冊子の目的

Kotlinの文法だけでなく、Spring Bootの各部品の役割と、Browser → Controller → Service → Repository → DB → View/APIという処理の流れを説明できるようにする。

目次

独立した「資料の使い方」章は設けず、必要な学習上の注意を各章へ組み込んでいる。

章	内容
1	Kotlin基礎
2	Controller
3	Thymeleaf
4	Spring DI / Bean
5	Service
6	Repository / Spring Data JPA / クエリメソッド
7	Entity / ORM / JPA
8	DTO
9	DB・SQL・PostgreSQL
10	HTTP / REST API / fetch
11	実際の処理の流れ
12	よく出るアノテーション
13	Pagination
14	Spring Security / Login
15	非同期処理
16	プロジェクト構成と設定ファイル
17	Rails経験との対応関係
18	Claude Code / AIを使った学習と検証
19	総まとめ
付録	公式リファレンス・用語・最終チェック

1. Kotlin基礎

fun : 関数

funで関数を定義する。引数には型、戻り値には:型を書く。

```
fun sum(x: Int, y: Int): Int {  
    return x + y  
}  
  
fun main() {  
    println(sum(1, 2))  
}
```

val / var

valは再代入不可、varは再代入可能。基本的にはvalを優先する。

```
val name = "Tanaka"  
  
var count = 10  
count = 20
```

型と型推論

Kotlinは静的型付け。初期値から型が明らかな場合は型を省略できる。

```
val age: Int = 25  
val name: String = "Tanaka"  
  
val price = 1000  
val message = "Hello"
```

List

listOfは読み取り専用List、mutableListOfは変更可能なMutableList。

```
val names = listOf("田中", "佐藤", "鈴木")  
  
for (name in names) {  
    println(name)  
}  
  
val mutableNames = mutableListOf("田中", "佐藤")  
mutableNames.add("鈴木")
```

if / when

ifは条件式、whenは複数分岐を整理する式。

```
val age = 25  
  
val result = if (age >= 20) {  
    "成人"  
} else {  
    "未成年"  
}  
  
val score = 80  
val rank = when {  
    score >= 90 -> "A"  
    score >= 70 -> "B"  
    score >= 50 -> "C"  
    else -> "D"  
}
```

for

コレクションや範囲を繰り返す。

```
for (i in 1..5) {  
    println(i)  
}  
  
for (i in 0 until 5) {  
    println(i)  
}
```

class

Kotlinでは主コンストラクタをクラス宣言へ直接書ける。

```
class User(  
    val id: Long,  
    val name: String  
)  
  
val user = User(1, "田中")  
println(user.name)
```

data class

データ保持に適したクラス。toString、equals、hashCode、copyなどが自動生成される。DTOでよく使う。

```
data class UserResponse(  
    val id: Long,  
    val name: String,  
    val email: String  
)
```

Null Safety

nullを許容する型はString?。?.はセーフコール、?:はエルビス演算子。

```
var name: String? = "Kotlin"  
name = null  
  
val length = name?.length ?: 0
```

2. Controller

@Controller

HTTPリクエストを受け、Serviceを呼び、ビュー名を返す画面向けController。

```
package com.example.app.controller

import org.springframework.stereotype.Controller
import org.springframework.ui.Model
import org.springframework.web.bind.annotation.GetMapping

@Controller
class UserController(
    private val userService: UserService
) {
    @GetMapping("/users")
    fun list(model: Model): String {
        val users = userService.findUsers()
        model.addAttribute("users", users)
        return "users/list"
    }
}
```

@RestController

戻り値をHTTPレスポンスボディとして扱うAPI向けController。JSONなどヘシリアライズされる。

```
package com.example.app.controller

import org.springframework.web.bind.annotation.GetMapping
import org.springframework.web.bind.annotation.RestController

@RestController
class UserApiController {
    @GetMapping("/api/users")
    fun list(): List<String> {
        return listOf("田中", "佐藤")
    }
}
```

リクエスト値

@RequestParamはクエリパラメータ等、@PathVariableはURLパス中の値、@RequestBodyはJSON等のボディ、@ModelAttributeはフォーム等をオブジェクトへバインドする。

```
@GetMapping("/users/search")
fun search(@RequestParam name: String): String {
    return "users/list"
}

@GetMapping("/users/{id}")
fun detail(@PathVariable id: Long): String {
    return "users/detail"
}

@PostMapping("/users")
fun create(@ModelAttribute form: UserCreateForm): String {
    userService.create(form)
    return "redirect:/users"
}
```

3. Thymeleaf

役割

ThymeleafはHTMLテンプレートエンジン。ControllerのModelをtemplates配下のHTMLへ埋め込む。RailsのERBに近い。

```
src/main/resources/
├── templates/
│   ├── index.html
│   └── users/
│       └── list.html
├── static/
│   ├── css/
│   ├── js/
│   └── images/
```

Model → HTML

ControllerでModelへ値を入れ、th:textなどで表示する。

```
// Controller
model.addAttribute("name", "田中")
return "index"

<!-- templates/index.html -->
<h1 th:text="${name}">名前</h1>
```

繰り返し・条件・リンク・フォーム

th:each、th:if、th:href、th:actionなどを使う。

```
<li th:each="user : ${users}"
    th:text="${user.name}">
    ユーザー名
</li>

<p th:if="${users.isEmpty()}">
    ユーザーがいません。
</p>

<a th:href="@{/users/{id}(id=${user.id})}">
    詳細
</a>

<form th:action="@{/users}" method="post">
    <input type="text" name="name">
    <button type="submit">登録</button>
</form>
```

4. Spring DI / Bean

Bean

Springが生成・管理するオブジェクトをBeanと呼ぶ。@Controller、@Service、@RepositoryなどがSpring管理対象になる。

constructor injection

Springが管理する依存Beanをコンストラクタへ注入する。Kotlinでは単一コンストラクタなら@Autowiredを省略できる。

```
@Service
class UserService(
    private val userRepository: UserRepository
) {
    fun findUsers(): List<User> {
        return userRepository.findAll()
    }
}
```

@Autowired

明示的に書くこともできるが、単一コンストラクタなら通常は不要。

```
@Service
class UserService @Autowired constructor(
    private val userRepository: UserRepository
) {
    // ...
}
```

5. Service

役割

業務処理・ユースケースを担当し、ControllerとRepositoryの間をつなぐ。

```
package com.example.app.service

import org.springframework.stereotype.Service

@Service
class UserService(
    private val userRepository: UserRepository
) {
    fun findUsers(): List<User> {
        return userRepository.findAll()
    }

    fun createUser(name: String, email: String): User {
        val user = User(
            name = name,
            email = email
        )
        return userRepository.save(user)
    }
}
```

責務分離

ControllerはHTTP、Serviceは業務処理、RepositoryはDBアクセス、Entityは永続化データ、DTOはデータ受け渡し、と考える。

6. Repository / Spring Data JPA / クエリメソッド

JpaRepository

JpaRepositoryのUserはEntity型、Longは主キー型。CRUDやページングを利用できる。

```
package com.example.app.repository

import com.example.app.model.User
import org.springframework.data.jpa.repository.JpaRepository

interface UserRepository : JpaRepository<User, Long> {
}
```

CRUD

findAll、findById、save、deleteById、existsByIdなどを利用できる。

```
val allUsers = userRepository.findAll()
val user = userRepository.findById(1L).orElse(null)
val saved = userRepository.save(user)
userRepository.deleteById(1L)
val exists = userRepository.existsById(1L)
```

メソッド名から検索条件

基本は find / exists / count / delete + By + Entityのプロパティ + 条件キーワード。Byが条件部分の開始。

```
interface UserRepository : JpaRepository<User, Long> {
    fun findByName(name: String): List<User>
    fun findByEmail(email: String): User?
    fun findByNameAndEmail(
        name: String,
        email: String
    ): List<User>
    fun findByNameContaining(keyword: String): List<User>
    fun findByAgeGreaterThan(age: Int): List<User>
    fun findByNameIgnoreCase(name: String): List<User>
    fun findByNameOrderByAgeDesc(name: String): List<User>
}
```

キーワード

And / Or / Containing / StartingWith / EndingWith / GreaterThan / GreaterThanEqual / LessThan / LessThanEqual / Between / In / IsNull / IgnoreCase / OrderBy / Distinct / Top / Firstなどがある。

```
findByNameAndEmail
findByNameContaining
findByAgeGreaterThan
findByAgeBetween
findByIdIn
findByEmailIsNull
findByNameIgnoreCase
findByNameOrderByAgeDesc
findDistinctByName
findTop10ByName
```

@Query

メソッド名だけでは表現しにくいクエリを明示する。通常はJPQL。

```
package com.example.app.repository

import com.example.app.model.User
import org.springframework.data.jpa.repository.JpaRepository
import org.springframework.data.jpa.repository.Query
import org.springframework.data.repository.query.Param

interface UserRepository : JpaRepository<User, Long> {
    @Query("select u from User u where u.name = :name")
    fun searchByName(
        @Param("name") name: String
    ): List<User>
}
```

公式参照

Spring Data JPA Query Keywords ReferenceとJPA Query

Methodsを、メソッド名の作り方を調べる一次資料として使う。

7. Entity / ORM / JPA

Entity

@EntityでJPAの永続化対象、@Idで主キー、@GeneratedValueでID生成方法を指定する。

```
package com.example.app.model

import jakarta.persistence.Entity
import jakarta.persistence.GeneratedValue
import jakarta.persistence.GenerationType
import jakarta.persistence.Id

@Entity
class User(
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    val id: Long? = null,
    val name: String,
    val email: String
)
```

JPA / Hibernate / Spring Data JPA

JPAは仕様、Hibernateは代表的な実装、Spring Data JPAはRepository抽象化などでJPAを扱いやすくする仕組み。

```
Application code
↓
Spring Data JPA
↓
JPA
↓
Hibernate
↓
SQL
↓
PostgreSQL
```

8. DTO

Request / Response DTO

外部から受け取るデータと外部へ返すデータを明示する。

```
data class UserCreateRequest(  
    val name: String,  
    val email: String  
)  
  
data class UserResponse(  
    val id: Long,  
    val name: String,  
    val email: String  
)
```

Entity → DTO

EntityをそのままAPIへ公開せず、DTOへ変換することで外部仕様とDBモデルを分離しやすい。

```
fun User.toResponse(): UserResponse {  
    return UserResponse(  
        id = requireNotNull(id),  
        name = name,  
        email = email  
    )  
}
```

9. DB ・ SQL ・ PostgreSQL

SQLの基本

CREATE / INSERT / SELECT / UPDATE / DELETEを理解する。

```
CREATE TABLE users (  
  id BIGSERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(255) NOT NULL  
);  
  
INSERT INTO users (name, email)  
VALUES ('田中', 'tanaka@example.com');  
  
SELECT * FROM users;  
  
SELECT * FROM users  
WHERE name = '田中';  
  
UPDATE users  
SET email = 'new@example.com'  
WHERE id = 1;  
  
DELETE FROM users  
WHERE id = 1;
```

Springとの関係

Repository → JPA/Hibernate → SQL → PostgreSQL。JPAを使っても最終的にはSQLとしてDBへ届く。

SQLファイル / psql

schema.sqlやdata.sqlでDB構造・初期データを管理し、psqlで確認できる。

```
-- schema.sql  
CREATE TABLE users (  
  id BIGSERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(255) NOT NULL  
);  
  
-- data.sql  
INSERT INTO users (name, email)  
VALUES ('田中', 'tanaka@example.com');  
  
-- psql  
\ dt  
SELECT * FROM users;  
\ q
```

10. HTTP / REST API / fetch

HTTPメソッド

GET=取得、POST=作成・送信、PUT/PATCH=更新、DELETE=削除が基本。

画面表示

GET /users → Controller → Service → Repository → PostgreSQL → Thymeleaf → HTML → Browser。

REST API

HTMLではなくレスポンスボディとしてデータを返す。JSONなどが一般的。

```
GET /api/users
```

```
[  
  {"id": 1, "name": "田中"},  
  {"id": 2, "name": "佐藤"}  
]
```

fetch

ブラウザ側JavaScriptからAPIを非同期で呼ぶ。

```
fetch("/api/users")  
  .then(response => response.json())  
  .then(users => {  
    renderUsers(users)  
  })
```

11. 実際の処理の流れ

一覧画面

HTTPからDB、そしてHTMLまでを一続きで追う。

```
Browser
↓ GET /users
Controller
↓
Service
↓
Repository
↓
JPA / Hibernate
↓ SQL
PostgreSQL
↓
Repository → Service → Controller
↓
Model
↓
Thymeleaf
↓
HTML
↓
Browser
```

フォーム登録

POST → Controller → Form/DTO → Service → Repository.save → INSERT → PostgreSQL → redirect。

```
Browser
↓ POST /users
Controller
↓ Form / DTO
Service
↓ 業務処理
Repository
↓ save()
JPA / Hibernate
↓ INSERT
PostgreSQL
↓
Controller
↓ redirect
Browser
```

API

Browser / JavaScript → @RestController → Service → Repository → DB → JSON → JavaScript → 画面更新。

12. よく出るアノテーション

一覧

@SpringBootApplication / @Controller / @RestController / @GetMapping / @PostMapping / @RequestParam / @PathVariable / @RequestBody / @ModelAttribute / @ResponseBody / @Service / @Repository / @Entity / @Id / @GeneratedValue / @Autowired / @Transactional / @Async。

Application.kt

Spring Bootアプリケーションの起点。

```
package com.example.app

import org.springframework.boot.autoconfigure.SpringBootApplication
import org.springframework.boot.runApplication

@SpringBootApplication
class Application

fun main(args: Array<String>) {
    runApplication<Application>(*args)
}
```


13. Pagination

基本

大量データをページ単位で取得する。Pageable、Page、Slice、Sort、PageRequestを使う。

```
interface UserRepository : JpaRepository<User, Long> {  
    fun findByNameContaining(  
        keyword: String,  
        pageable: Pageable  
    ): Page<User>  
}
```

Service

page番号と1ページ件数からPageableを作る。

```
fun findUsers(page: Int): Page<User> {  
    val pageable = PageRequest.of(page, 20)  
    return userRepository.findAll(pageable)  
}
```

Controller

HTTPのpageパラメータを受け取りServiceへ渡す。

```
@GetMapping("/users")  
fun list(  
    @RequestParam(defaultValue = "0") page: Int,  
    model: Model  
): String {  
    val users = userService.findUsers(page)  
    model.addAttribute("users", users)  
    return "users/list"  
}
```

14. Spring Security / Login

役割

認証・認可、アクセス制御、セッション、CSRF対策、パスワードのハッシュ化・検証などを扱う。

依存関係

spring-boot-starter-securityを追加する。

```
dependencies {  
    implementation("org.springframework.boot:spring-boot-starter-security")  
}
```

ログインの流れ

未認証なら/loginへ誘導し、認証後に保護されたURLへ進む。

```
Browser  
↓ GET /users  
Spring Security  
↓ 未認証  
/login  
↓  
ログイン  
↓  
認証済み  
↓  
/users  
↓  
Controller → Service → Repository → DB
```

認証と認可

Authenticationは誰かを確認、Authorizationは何をしてよいかを決める。

15. 非同期処理

ブラウザ側

fetchはブラウザ側の非同期通信。

@Async

Spring側の非同期メソッド実行。fetchとは別の仕組み。

```
import org.springframework.scheduling.annotation.Async
import org.springframework.stereotype.Service

@Service
class NotificationService {
    @Async
    fun sendNotification() {
        // 時間のかかる処理
    }
}
```

Spring MVC Web非同期

DeferredResult、Callable、WebAsyncTaskなどもある。@Asyncとは目的が異なる。

16. プロジェクト構成と設定ファイル

構成

main/kotlinはKotlin、resources/templatesはThymeleaf、resources/staticはCSS/JS/画像。

```
my-app/  
├── build.gradle.kts  
├── settings.gradle.kts  
├── src/  
│   ├── main/  
│   │   ├── kotlin/com/example/app/  
│   │   │   ├── Application.kt  
│   │   │   ├── controller/  
│   │   │   ├── service/  
│   │   │   ├── repository/  
│   │   │   ├── model/  
│   │   │   └── dto/  
│   │   └── resources/  
│   │       ├── templates/  
│   │       ├── static/  
│   │       │   ├── css/  
│   │       │   ├── js/  
│   │       │   └── images/  
│   │       └── application.yml  
│   └── test/kotlin/
```

build.gradle.kts

依存ライブラリを定義する。

```
dependencies {  
    implementation("org.springframework.boot:spring-boot-starter-web")  
    implementation("org.springframework.boot:spring-boot-starter-thymeleaf")  
    implementation("org.springframework.boot:spring-boot-starter-data-jpa")  
    runtimeOnly("org.postgresql:postgresql")  
    testImplementation("org.springframework.boot:spring-boot-starter-test")  
}
```

application.yml

DB接続やJPAなどの設定を記述する。

```
spring:  
  datasource:  
    url: jdbc:postgresql://localhost:5432/sample_db  
    username: postgres  
    password: password  
  jpa:  
    hibernate:  
      ddl-auto: update  
      show-sql: true  
  thymeleaf:  
    cache: false
```

17. Rails経験との対応関係

対応表

完全な1対1ではないが、既存知識との橋渡しに使える。

routes.rb → Mapping

routes.rbに近い役割を@GetMapping / @PostMappingなどが担う。

ERB → Thymeleaf

HTMLテンプレートという役割が近い。

Active Record → JPA / Spring Data JPA

ORM・DBアクセスという役割が近い。

database.yml → application.yml

設定ファイルとして対応させて考えられる。

Gemfile → build.gradle.kts

依存ライブラリ・ビルド設定として対応させられる。

注意

SpringではController → Service → Repository → Entityのように責務を明示的に分ける構成がよく使われる。

18. Claude Code / AIを使った学習と検証

学習サイクル

疑問・エラー → 仮説 → 公式Docs/教材/AIで調査 → 既存知識と対応 → 実装 → 実行・検証 → 新しい疑問。

使い分け

最初は公式チュートリアルを基準にして説明を求める。実装中は定型コードやUIの補助に使い、検証ではエラー原因・仕様との差分・テスト観点を確認する。

AIへの指示例

コードを別の書き方へ勝手に変更せず、処理の流れを説明させる。

このプロジェクトは Kotlin + Spring Boot
+ Thymeleaf + PostgreSQL です。

私はSpring Bootを学習中です。
コードを別の書き方へ書き換えないでください。

Controller → Service → Repository → DB の
処理の流れを、各行が何をしているか説明してください。

まず概念を説明し、その後にコード上の該当箇所を示してください。

教材を固定

KotlinやSpringには複数の正しい書き方がある。学習中は一つの公式教材・一つの書き方を基準にし、AIによる不要なスタイル変更を避ける。

19. 総まとめ：コードを読むときの視点

6つの質問

- ①このクラスは何層？ ②アノテーションは何を伝える？ ③引数はどこから？ ④戻り値は何？ ⑤次にどこへ？
⑥最終的にDB/HTML/JSONのどこへ？

典型的なController

HTTP入口 → Service → Model → Viewという流れで読む。

```
@GetMapping("/users")
fun list(
    @RequestParam(defaultValue = "0") page: Int,
    model: Model
): String {
    val users = userService.findUsers(page)
    model.addAttribute("users", users)
    return "users/list"
}
```

全体像

Browser → Controller → Service → Repository → JPA/Hibernate → PostgreSQL → Repository → Service → Controller → Thymeleaf → HTML → Browser。APIの場合はHTMLの代わりにJSONなどのHTTPレスポンス。

付録A：公式リファレンス

最終確認は公式ドキュメントへ戻す。特にRepositoryのメソッド名ルールは公式キーワード一覧を参照する。

Kotlin Tour： <https://kotlinlang.org/docs/kotlin-tour-welcome.html>

Spring Boot + Kotlin： <https://spring.io/guides/tutorials/spring-boot-kotlin/>

Spring Annotated Controllers： <https://docs.spring.io/spring-framework/reference/7.1/web/webmvc/mvc-controller.html>

@ModelAttribute： <https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller/ann-methods/modelattrib-method-args.html>

Dependency Injection： <https://docs.spring.io/spring-framework/reference/core/beans/dependencies/factory-collaborators.html>

Thymeleaf + Spring： <https://www.thymeleaf.org/doc/tutorials/3.1/thymeleafspring.html>

Spring Data JPA： <https://docs.spring.io/spring-data/jpa/reference/jpa.html>

JPA Query Methods： <https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html>

Query Keywords： <https://docs.spring.io/spring-data/jpa/reference/repositories/query-keywords-reference.html>

付録B：用語ミニ辞典

用語	意味
Spring Boot	Springアプリケーションを作りやすくするフレームワーク
Spring MVC	HTTPリクエストをController等で処理するWebフレームワーク
Bean	Springが生成・管理するオブジェクト
DI	依存オブジェクトを外部から注入する仕組み
Thymeleaf	HTMLテンプレートエンジン
JPA	永続化を扱う標準仕様
Hibernate	代表的なJPA実装
Spring Data JPA	JPAをRepository抽象化などで扱いやすくする仕組み
ORM	オブジェクトとリレーショナルDBの対応付け
Entity	JPAの永続化対象クラス
DTO	データ受け渡し用オブジェクト
REST API	HTTPを利用してデータを操作するAPI設計の代表的な考え方
Pagination	データをページ単位で分割して取得する仕組み
Authentication	誰であるかを確認すること
Authorization	何をしてよいかを決めること

付録C：最終チェックリスト

- ☐ Kotlin基礎：fun / val / var / 型 / List / if / when / for / class / data class / Null Safety
- ☐ Controller：@Controller / @RestController / @GetMapping / @PostMapping / @RequestParam / @PathVariable / @RequestBody / @ModelAttribute / @ResponseBody
- ☐ Thymeleaf：templates / Model / th:text / th:each / th:if / th:href / th:action / フォーム
- ☐ Spring DI / Bean：Bean / DI / constructor injection / @Autowired
- ☐ Service：業務処理・Controllerとの責務分離
- ☐ Repository：JpaRepository / CRUD / query derivation / @Query / Pagination
- ☐ クエリメソッド名ルール：find・exists・count・delete / By / property / And・Or / 条件キーワード / OrderBy / IgnoreCase / Top・First / Distinct
- ☐ JPA Repositoryの公式キーワード参照先
- ☐ Entity：@Entity / @Id / @GeneratedValue / ORM / JPA / Hibernate
- ☐ DTO：Request DTO / Response DTO / Entityとの分離
- ☐ DB・SQL：CREATE / INSERT / SELECT / UPDATE / DELETE / PostgreSQL
- ☐ SQLファイル：schema.sql / data.sql / psql
- ☐ HTTP / REST API：GET / POST / PUT / PATCH / DELETE / JSON
- ☐ fetchによるブラウザ側非同期通信
- ☐ 実際の処理の流れ：画面表示 / フォーム登録 / API
- ☐ よく出るアノテーション一覧
- ☐ Pagination：Pageable / Page / Slice / PageRequest
- ☐ Spring Security：認証 / 認可 / Login / CSRF等
- ☐ 非同期処理：fetch / @Async / Spring MVC Web非同期の違い
- ☐ プロジェクト構成：build.gradle.kts / src/main/kotlin / templates / static / application.yml / test
- ☐ Railsとの対応関係
- ☐ Claude Code / AIを使った学習と検証
- ☐ コードを読むための6つの質問と全体処理フロー
- ☐ 独立した「資料の使い方」章は作成していない
- ☐ 主要コード例はimport・class・関数を含めて掲載
- ☐ 印刷用に色数を抑え、コードブロックを薄いグレー背景＋等幅フォントで統一